Bayesian statistics with R
6. Bayesian analyses in R with the Jags software

Olivier Gimenez

November-December 2026

Bayes in practice

Software implementation (R friendly)

Oldies but goodies:

- WinBUGS, OpenBUGS: Where it all began.
- Jags: What we will use in this course.

Software implementation (R friendly)

Oldies but goodies:

- WinBUGS, OpenBUGS: Where it all began.
- Jags: What we will use in this course.

The new kids on the block:

- Nimble: What I'm going for these days.
- Stan: Entirely different algorithmic approach.
- Greta: Dunno anything about it.

Software implementation (R friendly)

Oldies but goodies:

- WinBUGS, OpenBUGS: Where it all began.
- Jags: What we will use in this course.

The new kids on the block:

- Nimble: What I'm going for these days.
- Stan: Entirely different algorithmic approach.
- Greta: Dunno anything about it.

If you're not into coding:

- brms: Bayesian regression models with Stan.
- MCMCglmm: Generalised Linear Mixed Models.
- Check out the CRAN Task View: Bayesian Inference for more.

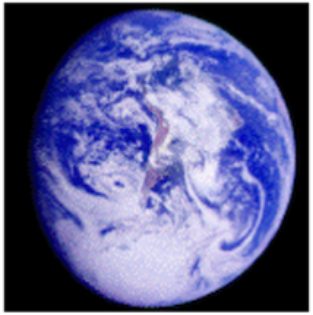
Introduction to JAGS (Just Another Gibbs Sampler)

Martyn Plummer



Real example

Impact of climatic conditions on white stork breeding success





Let's do a logistic regression on some White stork data

- Assess effects of temperature and rainfall on productivity.

Let's do a logistic regression on some White stork data

- Assess effects of temperature and rainfall on productivity.
- We have collected data.

Let's do a logistic regression on some White stork data

- Assess effects of temperature and rainfall on productivity.
- We have collected data.
- We need to build a model - write down the likelihood.

Let's do a logistic regression on some White stork data

- Assess effects of temperature and rainfall on productivity.
- We have collected data.
- We need to build a model - write down the likelihood.
- We need to specify priors for parameters.

Read in the data

```
nbchicks <- c(151,105,73,107,113,87,77,108,118,122,112,120,122,89,69,71,
              53,41,53,31,35,14,18)

nbpairs <- c(173,164,103,113,122,112,98,121,132,136,133,137,145,117,
             90,80,67,54,58,39,42,23,23)

temp <- c(15.1,13.3,15.3,13.3,14.6,15.6,13.1,13.1,15.0,11.7,15.3,14.4,
          14.4,12.7,11.7,11.9,15.9,13.4,14.0,13.9,12.9,15.1,13.0)

rain <- c(67,52,88,61,32,36,72,43,92,32,86,28,57,55,66,26,28,96,48,90,86,
          78,87)
```

```
datax <- list(N = 23,
              nbchicks = nbchicks,
              nbpairs = nbpairs,
              temp = (temp - mean(temp))/sd(temp),
              rain = (rain - mean(rain))/sd(rain))
```

Write down the model

$$\begin{aligned}
\text{nbchicks}_i &\sim \text{Binomial}(\text{nbpairs}_i, p_i) && \text{[likelihood]} \\
\text{logit}(p_i) &= a + b_{temp}\ \text{temp}_i + b_{rain}\ \text{rain}_i && \text{[linear model]} \\
a &\sim \text{Normal}(0, 1.5) && \text{[prior for } a\text{]} \\
b_{temp} &\sim \text{Normal}(0, 1.5) && \text{[prior for } b_{temp}\text{]} \\
b_{rain} &\sim \text{Normal}(0, 1.5) && \text{[prior for } b_{rain}\text{]}
\end{aligned}$$

Build the model

```
{
# Likelihood
    for( i in 1 : N){
        nbchicks[i] ~ dbin(p[i],nbpairs[i])
        logit(p[i]) <- a + b.temp * temp[i] + b.rain * rain[i]
        }
# ...
```

Specify priors

```
# Priors
a ~ dnorm(0,1/1.5)
b.temp ~ dnorm(0,1/1.5)
b.rain ~ dnorm(0,1/1.5)
}
```

Warning: Jags uses precision for Normal distributions (1 / variance)

You need to write everything in a file

```
model <-
paste("
model
{
    for( i in 1 : N)
        {
        nbchicks[i] ~ dbin(p[i],nbpairs[i])
        logit(p[i]) <- a + b.temp * temp[i] + b.rain * rain[i]
        }
a ~ dnorm(0,1/1.5)
b.temp ~ dnorm(0,1/1.5)
b.rain ~ dnorm(0,1/1.5)
    }
")
```

Alternatively, you may write a R function

```
logistic <- function() {
    for( i in 1 : N)
        {
        nbchicks[i] ~ dbin(p[i],nbpairs[i])
        logit(p[i]) <- a + b.temp * temp[i] + b.rain * rain[i]
        }

# priors for regression parameters
a ~ dnorm(0,1/1.5)
b.temp ~ dnorm(0,1/1.5)
b.rain ~ dnorm(0,1/1.5)
    }
```

Let us specify a few additional things

```
# list of lists of initial values (one for each MCMC chain)
init1 <- list(a = -0.5, b.temp = -0.5, b.rain = -0.5)
init2 <- list(a = 0.5, b.temp = 0.5, b.rain = 0.5)
inits <- list(init1,init2)

# specify parameters that need to be estimated
parameters <- c("a","b.temp","b.rain")

# specify nb iterations for burn-in and final inference
nb.burnin <- 10000
nb.iterations <- 20000 # beware: nb.iterations includes nb.burnin!
```

Run Jags

```
# load R2jags
library(R2jags)
# run Jags
storks <- jags(data  = datax,
               inits = inits,
               parameters.to.save = parameters,
               #model.file = "code/logistic.txt",
               model.file = logistic, # if a function was written
               n.chains = 2,
               n.iter = nb.iterations,
               n.burnin = nb.burnin)
storks
```

Inspect parameter estimates

```
#> Compiling model graph
#>    Resolving undeclared variables
#>    Allocating nodes
#> Graph information:
#>    Observed stochastic nodes: 23
#>    Unobserved stochastic nodes: 3
#>    Total graph size: 183
#>
#> Initializing model
#> Inference for Bugs model at "code/logistic.txt", fit using jags,
#>  2 chains, each with 20000 iterations (first 10000 discarded)
#>  n.sims = 20000 iterations saved. Running time = 0.366 secs
#>          mu.vect sd.vect    2.5%     25%     50%     75%   97.5%  Rhat n.eff
#> a         1.550   0.056   1.439   1.512   1.549   1.587   1.660 1.001 20000
#> b.rain   -0.156   0.062  -0.274  -0.199  -0.156  -0.115  -0.035 1.002  1400
#> b.temp    0.032   0.059  -0.085  -0.008   0.032   0.073   0.149 1.002  1300
#> deviance 204.609   2.457 201.796 202.811 203.968 205.729 210.999 1.002  2000
```

Your turn: Practical 5

MCMC makes you queens and kings of the stats world

Get all values sampled from posteriors

```
res <- as.mcmc(storks) # convert outputs in a list
res <- rbind(res[[1]],res[[2]]) # put two MCMC lists on top of each other
head(res)
#>              a     b.rain     b.temp deviance
#> [1,] 1.578385 -0.2474795 0.03168255 204.1438
#> [2,] 1.577992 -0.2056491 0.06228520 202.6206
#> [3,] 1.573356 -0.1899581 0.06516964 202.2897
#> [4,] 1.576637 -0.1903180 0.10126236 203.3674
#> [5,] 1.558328 -0.1844701 0.05983948 201.9717
#> [6,] 1.557940 -0.1578808 0.03813779 201.6088
```

```
tail(res)
#>                  a     b.rain       b.temp deviance
#> [19995,] 1.620095 -0.1832553 -0.006256992 203.7697
#> [19996,] 1.642103 -0.1887201 -0.025935767 205.5459
#> [19997,] 1.648377 -0.1628938 -0.015015713 205.0548
#> [19998,] 1.613646 -0.1672569 -0.012799917 203.3787
#> [19999,] 1.592314 -0.1659669 -0.018027813 202.8420
#> [20000,] 1.580429 -0.1580538  0.028542684 201.8391
```

Compute a posteriori Pr(rain < 0)

```
# probability that the effect of rainfall is negative
mean(res[,'b.rain'] < 0)
#> [1] 0.99315
```

Compute a posteriori Pr(temp < 0)

```
# probability that the effect of temperature is negative
mean(res[,'b.temp'] < 0)
#> [1] 0.29365
```

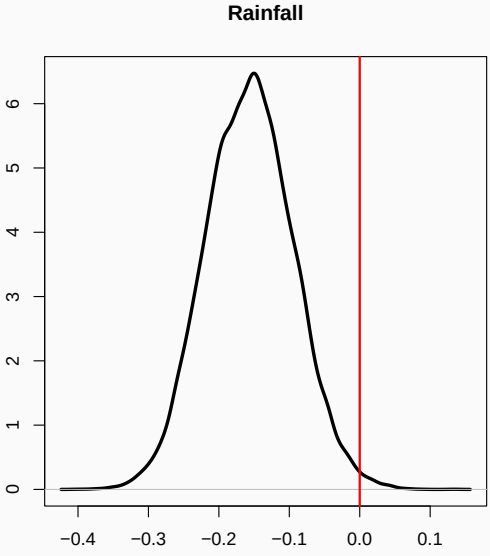
Get credible interval for the rain effect

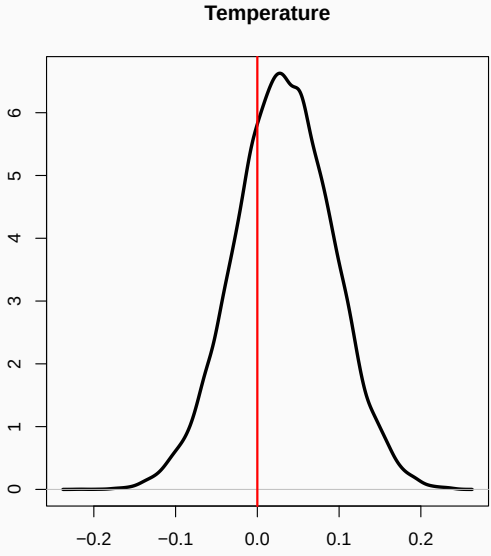
```
quantile(res[,'b.rain'],c(0.025,0.975))
#>        2.5%       97.5%
#> -0.27405842 -0.03489804
```

Get credible interval for the temperature effect

```
quantile(res[,'b.temp'],c(0.025,0.975))
#>         2.5%        97.5%
#> -0.08481843  0.14881264
```

Graphical summaries





Your turn: Practical 6